

Portfolio Technical Documentation

Architecture, Implementation, Deployment and Maintenance

Author	Mersie Ademova
System	Personal Engineering, Software Development and Research Portfolio
Website	https://mersiademova.github.io/mersiademova.github.io/index.html
Version	1.0
Date	August 2026

This document describes the public portfolio as published in August 2026.

Document Control

Version	Date	Author	Status
1.0	August 2026	Mersie Ademova	Initial public documentation

Purpose: to describe the architecture, technologies, content organization, deployment process, maintenance workflow, and technical limitations of the public portfolio website.

Contents

1. System Overview
2. Scope and Objectives
3. Technology Stack
4. Information Architecture and Page Structure
5. Portfolio Project Presentation Model
6. Front-End Architecture and User Interface
7. External Integrations
8. Search Engine Optimization and Discoverability
9. Deployment and Version Control
10. Testing and Quality Assurance
11. Security, Privacy and Data Handling
12. Maintenance Procedure
13. Known Limitations and Future Improvements
14. Appendix A: Public Page Inventory
15. Appendix B: Recommended Documentation Menu

1. System Overview

The portfolio is a public, static, multi-page website that presents the academic, engineering, software-development, research, and technical-writing work of Mersie Ademova. It is designed as a professional presentation platform rather than a transactional web application. The website combines profile information, a structured project portfolio, detailed case studies, academic and technical writings, external source-code repositories, and browser-accessible application demonstrations.

The system is published through GitHub Pages and is accessible through the following public address:

[Mersie Ademova Portfolio](#)

Architectural classification

Static front-end website with external links to GitHub repositories, Streamlit applications, social profiles, downloadable documents, and email contact.

2. Scope and Objectives

The portfolio is intended to provide a coherent and verifiable presentation of work across several connected areas. Its objectives are to:

- present a clear professional identity across engineering, software development, applied research, and technical authorship;
- organize projects by system type and analytical purpose;
- show the complete path from problem identification to architecture, implementation, testing, evaluation, and documentation;
- provide direct access to source code, live applications, academic papers, and case studies;
- support academic evaluation, professional review, collaboration, and future platform development;
- maintain a consistent, minimal and responsive visual language across all pages.

The website does not currently include user accounts, online payments, a content-management system, server-side data processing, or a private administrative interface.

3. Technology Stack

Layer / Component	Technology	Purpose
Document structure	HTML5	Defines the content and semantic structure of all public pages.
Styling	CSS3 and custom main.css	Provides typography, spacing, colors, responsive behavior, cards, buttons and page-specific layouts.
Responsive framework	Bootstrap 5.3.3	Supports the responsive grid, navigation, spacing utilities and reusable layout classes.
Client-side behavior	JavaScript	Initializes navigation behavior, animation libraries, sliders, filters and page utilities.

Layer / Component	Technology	Purpose
Icons	Bootstrap Icons	Provides interface and social-media icons.
Animation	AOS	Adds controlled scroll-based reveal animations.
Project filtering	Isotope and imagesLoaded	Supports category filtering and stable masonry-style project layout.
Image sliders	Swiper	Displays project screenshots in responsive carousels.
Media viewing	GLightbox	Supports enlarged viewing of images or linked media where enabled.
Hosting	GitHub Pages	Publishes the static website directly from the portfolio repository.
Version control	Git and GitHub	Tracks changes, stores source files and publishes revisions.
External application hosting	Streamlit Community Cloud	Hosts selected interactive Python applications linked from the portfolio.

4. Information Architecture and Page Structure

The website uses a persistent top navigation bar and a shared footer. The content is divided into primary profile pages, project pages, case studies, writings, documentation, and contact resources.

4.1 Primary navigation

- **Home** – introduces the professional identity and central portfolio statement.
- **About** – describes the author’s interdisciplinary approach and current areas of work.
- **Resume** – summarizes education, independent development, and professional experience.
- **What I Do** – organizes capabilities into software engineering, application development, systems analysis, computation, research, and technical writing.
- **Portfolio** – presents the main projects and supports category-based filtering.
- **Additional Writings** – presents academic papers, technical studies, specifications, and code-supported research.
- **Documentation** – provides the technical documentation and user guide for the portfolio itself.
- **Contact** – provides email, LinkedIn and GitHub contact channels.

4.2 Shared layout components

- sticky responsive header and mobile navigation toggle;
- consistent logo position and social links;
- section titles with short descriptive text;
- footer with copyright, social links and template attribution;
- automatic current-year display through JavaScript;
- shared vendor styles and scripts loaded from the assets directory.

4.3 Directory organization

The repository follows a conventional static-site structure. The exact list may evolve, but the principal organization is:

```

portfolio-root/
├─ index.html
├─ about.html
├─ resume.html
├─ services.html
├─ portfolio.html
├─ additional_writings.html
├─ contact.html
├─ project overview pages (*.html)
├─ project case-study pages (*-case_study.html)
├─ assets/
│   ├─ css/main.css
│   ├─ js/main.js
│   ├─ img/
│   └─ vendor/
├─ writings/
├─ documentation/
├─ sitemap.xml
└─ repository support files

```

5. Portfolio Project Presentation Model

The Portfolio page groups projects into three broad categories: Software Systems, Embedded & Engineering Systems, and Computational Research & Experiments. The current public portfolio includes the Decision Support System, two RFID access-control prototypes, the Numerical Methods Solver, California Housing Price Prediction, the Information Theory Toolkit, QuenchMap, and the Parallel Sorting Benchmark.

5.1 Project overview page

- project title and concise technology line;
- image carousel or visual evidence;
- category, project type, date, status and author role;
- short project summary;
- links to the detailed case study, live application, source code and supporting document where available.

5.2 Case-study page

- problem and motivation;
- project objective;
- functional modules and mathematical or engineering foundations;
- architecture and workflow;
- testing and evaluation;
- outcome, strengths and limitations;
- future development;
- documentation and public project resources.

This two-level model prevents the main portfolio from becoming overloaded while still allowing serious reviewers to inspect the reasoning, implementation and evidence behind each project.

6. Front-End Architecture and User Interface

6.1 Responsive design

Bootstrap grid classes and custom media queries adapt the layout to desktop, tablet and mobile widths. Content cards, project images, case-study navigation, action buttons and architecture grids are designed to wrap or stack when screen width decreases.

6.2 Visual system

- minimal white and light-background interface;
- turquoise-green accent color used for active states, buttons and structural highlights;
- Poppins, Roboto and Raleway web fonts;
- consistent rounded cards, restrained shadows and clear spacing;
- visual evidence from real interfaces, charts, hardware prototypes and documents rather than decorative stock imagery.

6.3 Interaction patterns

- responsive navigation and dropdown menus;
- portfolio category filters;
- project screenshot sliders;
- call-to-action buttons with clear destination labels;
- external links opened in a new browser tab where appropriate;
- scroll-to-top control and subtle section animations.

7. External Integrations

Layer / Component	Technology	Purpose
GitHub	Public repositories and profile	Provides source code, README files, version history and implementation evidence.
Streamlit Community Cloud	Live Python web applications	Provides browser-accessible demonstrations of selected analytical and computational projects.
LinkedIn	Professional profile	Supports professional identity and communication.
Email	mailto link	Primary method for detailed professional inquiries.
Facebook and Instagram	Social links	Additional public contact and profile channels.
Google Fonts	Remote font delivery	Loads the typography used by the site.

Availability note

External repositories, live applications and social profiles depend on third-party services. A temporary failure of an external service does not indicate failure of the static portfolio website itself.

8. Search Engine Optimization and Discoverability

- descriptive page titles and project-specific meta descriptions;
- keyword metadata on project and case-study pages;
- semantic heading structure;
- descriptive project text and visible page titles;
- favicon and Apple touch icon;
- sitemap.xml for search-engine discovery;
- public GitHub Pages URL and direct internal linking between pages;
- meaningful link labels such as View Case Study, Live Web Application, Source Code and View Document.

SEO metadata should be reviewed whenever a project is renamed, a new page is added, or the professional positioning of the portfolio changes.

9. Deployment and Version Control

The portfolio is maintained locally and published through GitHub. The standard deployment workflow is:

```
cd "E:\Portfolio1"
git status
git add .
git commit -m "Describe the portfolio update"
git push
```

GitHub Pages then publishes the updated files from the configured branch. If the remote repository contains changes that are not available locally, the repository should be synchronized before pushing:

```
git pull --rebase origin main
git push
```

Deployment rule

Every public link should be tested after deployment, especially case-study links, PDF paths, GitHub repository links and Streamlit application URLs.

10. Testing and Quality Assurance

The portfolio should be treated as a software product and checked after meaningful updates. Recommended verification includes:

- open every top-level navigation item;
- verify dropdown behavior on desktop and mobile;
- test portfolio filters and image sliders;
- open every project overview and case-study page;
- test all Live Application, Source Code, View Document and README links;
- verify that PDFs open and use the correct path and filename;
- check responsive layout at desktop, tablet and mobile widths;

- inspect browser console for missing files or JavaScript errors;
- verify title, description and favicon on new pages;
- check spelling, spacing, terminology and date consistency;
- confirm that the public GitHub Pages version reflects the latest commit.

11. Security, Privacy and Data Handling

Because the portfolio is a static public website, all published HTML, images and documents should be considered publicly accessible. The website does not process credentials or store private user data. Nevertheless, public publication requires careful control of project content.

- do not publish passwords, tokens, private keys, confidential database files or internal organizational identifiers;
- use anonymized or partially synthetic data for public demonstrations when original operational data is confidential;
- avoid exposing real access-control UIDs or other sensitive hardware identifiers;
- review documents for personal data before upload;
- use `rel="noopener noreferrer"` for external links opened in a new tab;
- publish only contact information intentionally intended for public use.

12. Maintenance Procedure

12.1 Adding a new project

- prepare the repository and README where public source code is appropriate;
- prepare a live demonstration only when it adds real value;
- create a project overview page using the established visual structure;
- create or update the case-study page;
- add project images with meaningful filenames and alternative text;
- add the project card to `portfolio.html` and assign the correct category;
- add document links and external resources;
- update `sitemap.xml` and test all routes;
- commit and deploy the complete update as one coherent release.

12.2 Updating an existing project

- synchronize the project description with the current implementation;
- replace placeholder or outdated links;
- update status, technologies, testing results and limitations;
- verify that the case study does not describe future work that is already complete;
- keep terminology consistent across portfolio page, case study, README and academic document.

13. Known Limitations and Future Improvements

- the site currently depends on manual HTML updates rather than a content-management system;
- some project demonstrations depend on external hosting availability;
- the documentation menu is maintained manually across multiple HTML files;

- accessibility can be improved through a complete alt-text audit, keyboard-navigation review and contrast testing;
- structured data markup could improve search-engine interpretation;
- automated link checking and deployment validation could reduce maintenance effort;
- project screenshots and collaborator descriptions may be refined over time without changing the core architecture.

Appendix A: Public Page Inventory

Layer / Component	Technology	Purpose
Home	index.html	Professional introduction and central portfolio statement.
About	about.html	Background, approach and collaborators.
Resume	resume.html	Education and professional experience.
What I Do	services.html	Capabilities and areas of work.
Portfolio	portfolio.html	Project gallery and category filters.
Additional Writings	additional_writings.html	Academic papers, technical studies and specifications.
Contact	contact.html	Email, LinkedIn and GitHub contact routes.
Project overview pages	project-specific HTML files	Summary, screenshots, project data and action links.
Case studies	*-case_study.html	Detailed technical and analytical project presentation.
Documentation	dropdown links	Portfolio Technical Documentation and Portfolio User Guide.

Appendix B: Recommended Documentation Menu

A separate navigation item named Documentation should contain two dropdown links:

- **Technical Documentation** – documentation/portfolio-technical-documentation.pdf
- **User Guide** – documentation/portfolio-user-guide.pdf

The same documentation navigation should be added consistently to every top-level, project-overview and case-study page.